



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №0
по дисциплине «Системы анализа больших данных»

Студент группы *ИНБО-21-23. Илюхин К.Н.*

(подпись)

Преподаватель *Алексеева Е.С.*

(подпись)

Москва 2026 г.

СОДЕРЖАНИЕ

1 ВВЕДЕНИЕ.....	4
1.1 Цель и постановка задачи.....	4
2 ХОД РАБОТЫ	5
2.1 Установка Cassandra NoSQL	5
2.2 Работа в CQL shell	6
2.3 Доступ к данным с помощью DDL.....	8
2.4 Доступ к данным с помощью DML	11
2.5 Самостоятельная работа	16
ВЫВОДЫ	19

1 ВВЕДЕНИЕ

1.1 Цель и постановка задачи

Цель практической работы — освоение основных принципов работы с распределённой NoSQL базой данных Apache Cassandra, ориентированной на высокую доступность, горизонтальную масштабируемость и производительность при обработке больших объёмов данных. В ходе выполнения решаются задачи по установке системы, созданию и настройке ключевых пространств, проектированию таблиц с учётом особенностей партиционирования и кластеризации, а также выполнению операций языка CQL (DDL и DML). Работа направлена на понимание отличий модели данных Cassandra от реляционной, включая необходимость денормализации, правильный выбор ключей и ограничения запросов в распределённой среде.

2 ХОД РАБОТЫ

2.1 Установка Cassandra NoSQL

Для дальнейшей работы установим Cassandra NoSQLy, создадим Cassandra репозиторий (Рисунок 1).

```
[user@vbox ~]$ chfn user
Changing finger information for user.
Name [Kirill Ilyuhin]:
Office []:
Office Phone []:
Home Phone []:

Finger information not changed.
[user@vbox ~]$
[user@vbox ~]$ sudo tee /etc/yum.repos.d/cassandra.repo > /dev/null << 'EOF' [cassandra]
> name=Apache Cassandra
> baseurl=https://redhat.cassandra.apache.org/50x/
> gpgcheck=1
> repo_gpgcheck=1
> gpgkey=https://downloads.apache.org/cassandra/KEYS
> EOF

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

[sudo] password for user:
[user@vbox ~]$ _
```

Рисунок 1 — Создание репозитория для Cassandra

Теперь установим Cassandra NoSQL (Рисунок 2).

```
Installed:
  cassandra.noarch 0:4.1.12-1

Dependency Installed:
  avahi-libs.x86_64 0:0.6.31-20.el7
  cups-libs.x86_64 1:1.6.3-52.el7_9
  javapackages-tools.noarch 0:3.4.1-11.el7
  libtirpc.x86_64 0:0.2.4-0.16.el7
  lkctp-tools.x86_64 0:1.0.17-2.el7
  python-javapackages.noarch 0:3.4.1-11.el7
  python3.x86_64 0:3.6.8-21.el7_9
  python3-pip.noarch 0:9.0.3-8.el7
  tzdata-java.noarch 0:2024a-1.el7

copy-jdk-configs.noarch 0:3.3-11.el7_9
java-1.8.0-openjdk-headless.x86_64 1:1.8.0.412.b08-1.el7_9
libjpeg-turbo.x86_64 0:1.2.90-8.el7
libxslt.x86_64 0:1.1.28-6.el7
pcsc-lite-libs.x86_64 0:1.8.8-8.el7
python-lxml.x86_64 0:3.2.1-4.el7
python3-libs.x86_64 0:3.6.8-21.el7_9
python3-setuptools.noarch 0:39.2.0-10.el7

Complete!
[user@vbox ~]$
```

Рисунок 2 — Установка Cassandra NoSQL

Запустим соответствующие установленные сервисы (Рисунок 3).

```
complete:
[user@vbox ~]$ sudo systemctl daemon-reload
[user@vbox ~]$ sudo systemctl start cassandra
```

Рисунок 3 — Запуск сервисов

Проверим работу службы кассандра (Рисунок 4).

```
[user@vbox ~]$ sudo systemctl status cassandra
● cassandra.service - LSB: distributed storage system for structured data
   Loaded: loaded (/etc/rc.d/init.d/cassandra; bad; vendor preset: disabled)
   Active: active (running) since Tue 2026-09-01 12:18:47 EDT; 1min 22s ago
     Docs: man:systemd-sysv-generator(8)
  Process: 9526 ExecStart=/etc/rc.d/init.d/cassandra start (code=exited, status=0/SUCCESS)
 Main PID: 9629 (java)
   CGroup: /system.slice/cassandra.service
           └─9629 /usr/lib/jvm/java-1.8.0-openjdk-1.8.0.412.b08-1.el7_9.x86_64/jre/bin/java -ea -da:net.openhft... -X..

Sep 01 12:18:47 vbox systemd[1]: Starting LSB: distributed storage system for structured data...
Sep 01 12:18:47 vbox runuser[9536]: pam_unix(runuser:session): session opened for user cassandra by (uid=0)
Sep 01 12:18:47 vbox systemd[1]: Started LSB: distributed storage system for structured data.
Sep 01 12:18:47 vbox cassandra[9526]: Starting Cassandra: OK
[user@vbox ~]$
```

Рисунок 4 — запущенная Cassandra

В результате все соответствующие службы установлены и готовы к работе.

2.2 Работа в CQL shell

Запустим CQL shell (Рисунок 5).

```
[user@vbox ~]$ cqlsh

ATTENTION: All commands will be saved to history file: /home/user/.cassandra/cqlsh_history
This may include sensitive information such as passwords.
To disable history, use --disable-history or set 'disabled = true' in the [history] section of cqlshrc.
See https://cassandra.apache.org/doc/latest/tools/cqlsh.html for more information.

Connected to Test Cluster at 127.0.0.1:9042
[cqlsh 6.1.0 | Cassandra 4.1.12 | CQL spec 3.4.6 | Native protocol v5]
Use HELP for help.
cqlsh>
```

Рисунок 5 — CQL shell

Выведем список всех пространств ключей в кластере (Рисунок 6)

```
cqlsh> desc keyspaces;

system          system_distributed  system_traces  system_virtual_schema
system_auth     system_schema      system_views

cqlsh>
```

Рисунок 6 — Список ключей в кластере

Создадим пространство ключей с именем «ks_work1» на одном узле (Рисунок 7).

```
cqlsh> CREATE KEYSPACE ks_work1
...     WITH REPLICATION = {
...       'class' : 'SimpleStrategy',
...       'replication_factor' : 1
...     };
```

Рисунок 7 — Создание пространства ключей

Попробуем создать пространство ключей в тестовом кластере с одним центром обработки данных и одним узлом (Рисунок 8).

```
cqlsh> CREATE KEYSPACE ks_work2
...     WITH REPLICATION = {
...       'class' : 'NetworkTopologyStrategy',
...       'datacenter1' : 1
...     } ;
```

Рисунок 8 — Тестовый кластер

Просмотрим информацию о созданных пространствах ключей (Рисунок 9).

```
cqlsh> describe keyspace ks_work1;
CREATE KEYSPACE ks_work1 WITH replication = {'class': 'SimpleStrategy', 'replication_factor': '1'} AND durable_writes = true;
cqlsh> desc ks_work2;
CREATE KEYSPACE ks_work2 WITH replication = {'class': 'NetworkTopologyStrategy', 'datacenter1': '1'} AND durable_writes = true;
cqlsh>
```

Рисунок 9 — Информация о пространстве ключей

Попробуем изменить параметры пространства ключей (Рисунок 10).

```
cqlsh> alter keyspace ks_work2 with durable_writes = false;
cqlsh> desc ks_work2;
CREATE KEYSPACE ks_work2 WITH replication = {'class': 'NetworkTopologyStrategy', 'datacenter1': '1'} AND durable_writes = false;
cqlsh>
```

Рисунок 10 — Измененные параметры

Проверим пространство ключей после удаления (Рисунок 11).

```
cqlsh> drop KEYSPACE ks_work2;
cqlsh> desc ks_work2;
'ks_work2' not found in keyspaces
```

Рисунок 11 — Удаленное пространство ключей

Таким образом, была продемонстрирована работа с пространствами ключей и их изменением.

2.3 Доступ к данным с помощью DDL

Создадим таблицу со следующими параметрами (Рисунок 12):

- Keyspace: ks_work1;
- Table name: user_info;
- Column name: id uuid, l_name text, f_name text, country text, birth int, gender text;
- Partition Key: id.

```
cqlsh> DESC TABLES;

Keyspace ks_work1
-----
user_info

Keyspace system
-----
available_ranges      paxos_repair_history  sstable_activity
available_ranges_v2  peer_events           sstable_activity_v2
batches              peer_events_v2       table_estimates
built_views          peers                top_partitions
compaction_history   peers_v2             transferred_ranges
"IndexInfo"          prepared_statements  transferred_ranges_v2
local                repairs              view_builds_in_progress
paxos                size_estimates

Keyspace system_auth
-----
network_permissions      role_members      roles
resource_role_permissions_index  role_permissions

Keyspace system_distributed
-----
parent_repair_history  partition_denylist  repair_history  view_build_status

Keyspace system_schema
-----
aggregates  dropped_columns  indexes  tables  types
columns     functions        keyspaces  triggers  views
```

Рисунок 12 — Созданная таблица

Создадим еще одну таблицу со следующими параметрами:

- Table name: user_info2;
- Column name: id uuid, l_name text, f_name text, country text, birth int, gender text, s text, num int;
- static column: s;
- primary key : country, birth, id;

- composite partition key: country, birth.

```
cqlsh> DESC TABLES;

Keyspace ks_work1
-----
user_info  user_info2

Keyspace system
-----
available_ranges      paxos_repair_history  sstable_activity
available_ranges_v2   peer_events            sstable_activity_v2
batches               peer_events_v2        table_estimates
built_views           peers                  top_partitions
compaction_history    peers_v2               transferred_ranges
"IndexInfo"           prepared_statements    transferred_ranges_v2
local                 repairs                 view_builds_in_progress
paxos                 size_estimates

Keyspace system_auth
-----
network_permissions    role_members          roles
resource_role_permissions_index  role_permissions

Keyspace system_distributed
-----
parent_repair_history  partition_denylist    repair_history  view_build_status

Keyspace system_schema
-----
aggregates  dropped_columns  indexes  tables  types
columns     functions        keyspaces  triggers  views
```

Рисунок 13 — Создание второй таблицы

Создадим еще несколько таблиц (Рисунок 14).

```
cqlsh> create table ks_work1.user_info3 (id int, l_name text, f_name text, country text, birth text, gender text, s text,
static, num int, primary key(country, id, birth));
cqlsh> create table ks_work1.user_info33 (id int, l_name text, f_name text, country text, birth text, gender text, s text,
static, age int, primary key(id, birth)) with clustering order by (birth desc);
cqlsh> DESC TABLES;

Keyspace ks_work1
-----
user_info  user_info2  user_info3  user_info33

Keyspace system
-----
available_ranges      paxos_repair_history  sstable_activity
available_ranges_v2   peer_events            sstable_activity_v2
batches               peer_events_v2        table_estimates
built_views           peers                  top_partitions
compaction_history    peers_v2               transferred_ranges
"IndexInfo"           prepared_statements    transferred_ranges_v2
local                 repairs                 view_builds_in_progress
paxos                 size_estimates

Keyspace system_auth
-----
network_permissions    role_members          roles
resource_role_permissions_index  role_permissions

Keyspace system_distributed
-----
parent_repair_history  partition_denylist    repair_history  view_build_status
```

Рисунок 14 — Создание таблиц

Изменим пространство ключей и проверим созданы ли таблицы (Рисунок 15).

```

cqlsh> use ks_work1;
cqlsh:ks_work1> DESC TABLES;

user_info  user_info2  user_info3  user_info33

cqlsh:ks_work1> DESC USER_INFO;

CREATE TABLE ks_work1.user_info (
  id uuid PRIMARY KEY,
  birth int,
  country text,
  f_name text,
  gender text,
  l_name text
) WITH additional_write_policy = '99p'
  AND bloom_filter_fp_chance = 0.01
  AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
  AND cdc = false
  AND comment = ''
  AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
  AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
  AND memtable = 'default'
  AND crc_check_chance = 1.0
  AND default_time_to_live = 0
  AND extensions = {}
  AND gc_grace_seconds = 864000
  AND max_index_interval = 2048
  AND memtable_flush_period_in_ms = 0

```

Рисунок 15 — Проверка user_info

Добавим еще один столбец в таблицу и проверим информацию (Рисунок 16).

```

cqlsh:ks_work1> alter table user_info33 add height int;
cqlsh:ks_work1> DESC USER_INFO33;

CREATE TABLE ks_work1.user_info33 (
  id int,
  birth text,
  s text static,
  age int,
  country text,
  f_name text,
  gender text,
  height int,
  l_name text,
  PRIMARY KEY (id, birth)
) WITH CLUSTERING ORDER BY (birth DESC)
  AND additional_write_policy = '99p'
  AND bloom_filter_fp_chance = 0.01
  AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
  AND cdc = false
  AND comment = ''
  AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
  AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
  AND memtable = 'default'
  AND crc_check_chance = 1.0
  AND default_time_to_live = 0
  AND extensions = {}
  AND gc_grace_seconds = 864000
  AND max_index_interval = 2048
  AND memtable_flush_period_in_ms = 0

```

Рисунок 16 — Добавление столбца в таблицу

Удалим только что изменённую таблицу и перечислим таблицы, чтобы убедиться, что все корректно удалено (Рисунок 17).

```
cqlsh:ks_work1> drop table user_info33;  
cqlsh:ks_work1> DESC TABLES;  
  
user_info  user_info2  user_info3  
cqlsh:ks_work1>
```

Рисунок 17 — Оставшиеся таблицы

Введём данные в таблице user_info2 (Рисунок 18).

```
cqlsh:ks_work1> INSERT INTO user_info2(country, birth, id, s, f_name, gender, l_name, num) VALUES ('KOR', '1995', 1, 'value8', 'jason', 'M', 'jeong', 18);  
, '1995', 5, 'value8', 'lonan', 'M', 'jeong', 53);  
INSERT INTO user_info2(country, birth, id, s, f_name, gender, l_name, num) VALUES ('USA', '1967', 3, 'value6', cqlsh:ks_lue8', 'lonan', 'M', 'jeong', 53);try, birth, id, s, f_name, gender, l_name, num) VALUES ('KOR', '1995', 5, 'va  
'scott', 'M', 'jeong', 32);  
INSERT INTO user_info2(country, birth, id, s, f_name, gender, l_name, num) VALUES ('USA', '1967', 6, 'value6', 'angela', 'F', 'park', 25);  
INSERT INTO user_info2(country, birth, id, s, f_name, gender, l_name, num) VALUES ('USA', cqlsh:ks_work1> INSERT INTO uslue6', 'scott', 'M', 'jeong', 32);name, gender, l_name, num) VALUES ('USA', '1967', 3, 'v  
'1974', 4, 'value4', 'jane', 'F', 'null', 0);  
INSERT INTO user_info2(country, birth, id, s, f_name, gender, l_name, num) VALUES ('USA', '1985', 2, 'value2', 'tommy', 'M', 'park', 19);  
INSERT INTO user_info2(country, birth, id, s, f_name, gender, l_name, num) VALUES ('CAN', '1994', 7, 'value7', 'bella', 'F', 'null', 41);cqlsh:ks_work1> INSERT INTO user_info2(country, birth, id, s, f_name, ge  
der, l_name, num) VALUES ('USAlue6', 'angela', 'F', 'park', 25);  
cqlsh:ks_work1> INSERT INTO user_info2(country, birth, id, s, f_name, gender, l_name, num) VALUES ('USA', '1974', 4, 'value4', 'jane', 'F', 'null', 0);  
cqlsh:ks_work1> INSERT INTO user_info2(country, birth, id, s, f_name, gender, l_name, num) VALUES ('USA', '1985', 2, 'value2', 'tommy', 'M', 'park', 19);  
cqlsh:ks_work1> INSERT INTO user_info2(country, birth, id, s, f_name, gender, l_name, num) VALUES ('CAN', '1994', 7, 'value7', 'bella', 'F', 'null', 41);  
cqlsh:ks_work1>
```

Рисунок 18 — Добавление данных в таблицу

Аналогичным образом добавим информацию в таблицу user_info3 (Рисунок 19).

```
cqlsh:ks_work1> INSERT INTO user_info3(country, id, birth, s, f_name, gender, l_name, num) VALUES ('CAN', 7, '1994', 'value7', 'bella', 'F', NULL, 41);  
er, l_name, num) VALUES ('KOR', 1, '1995', 'value8', 'jason', 'M', 'jeong', 18);  
INSERT INTO user_info3(country, id, birth, s, f_name, gender, l_name, num) VALUES ('KOR', 5, '1995', 'value8', 'lonan', 'M', 'kim', 53);  
INSERT INTO user_info3(country, id, birth, s, f_name, gender, l_name, num) VALUES ('USA', 8, '1985', 'value8', 'tommy', 'F', NULL, 61);  
INSERT INTO user_info3(country, id, birth, s, f_name, gender, l_name, num) VALUES ('USA', 2, '1995', 'value8', 'kicqlsh:ks_work1> INSERT INTO user_info3(country, id, birth, s, f_name, gender, l_name, VAL  
UES ('KOR', 1, '1995', 'value8', 'jason', 'M', 'jeong', 18);  
s, 'F', NULL, 61);  
INSERT INTO user_info3(country, id, birth, s, f_name, gender, l_name, num) VALUES ('USA', 3, '1967', 'value6', 'scitttt', 'M', 'park', 32);  
INSERT INTO user_info3(country, id, birth, s, f_name, gender, l_name, num) VALUES ('USA', 6, '1967', 'value6', 'angela', 'F', 'park', 25);  
INSERT INTO user_info3(country, id, birth, s, f_name, gender, l_name, num) VALUES ('JPN', 4, '1974', 'value4', 'jane', cqlsh:ks_work1> INSERT INTO user_info3(country, id, birth, s, f_name, gender, l_name, num)  
, VALUES ('KOR', 5, '1995', 'value8', 'lonan', 'M', 'kim', 53);  
cqlsh:ks_work1> INSERT INTO user_info3(country, id, birth, s, f_name, gender, l_name, num) VALUES ('USA', 8, '1985', 'value8', 'tommy', 'F', NULL, 61);  
cqlsh:ks_work1> INSERT INTO user_info3(country, id, birth, s, f_name, gender, l_name, num) VALUES ('USA', 2, '1995', 'value8', 'kim', 'F', NULL, 61);  
'F', NULL, NULL);cqlsh:ks_work1> INSERT INTO user_info3(country, id, birth, s, f_name, gender, l_name, num) VALUES ('USA', 3, '1967', 'value6', 'scitttt', 'M', 'park', 32);  
cqlsh:ks_work1> INSERT INTO user_info3(country, id, birth, s, f_name, gender, l_name, num) VALUES ('USA', 6, '1967', 'value6', 'angela', 'F', 'park', 25);  
cqlsh:ks_work1> INSERT INTO user_info3(country, id, birth, s, f_name, gender, l_name, num) VALUES ('JPN', 4, '1974', 'value4', 'jane', 'F', NULL, NULL);  
cqlsh:ks_work1>
```

Рисунок 19 — Добавление данных в таблицу

В результате таблицу созданы, заполнены и готовы к дальнейшей работе с ними.

2.4 Доступ к данным с помощью DML

Проверим как записались данные из предыдущего пункта (Рисунок 20).

```
cqlsh:ks_work1> select * from user_info2
... ;
```

country	birth	id	f_name	gender	l_name	num	s
KOR	1995	1	jason	M	jeong	18	value8
KOR	1995	5	lonan	M	jeong	53	value8
USA	1967	3	scott	M	jeong	32	value6
USA	1967	6	angela	F	park	25	value6
USA	1974	4	jane	F	null	0	value4
USA	1985	2	tommy	M	park	19	value2
CAN	1994	7	bella	F	null	41	value7

(7 rows)

Рисунок 20 — Информация в user_info2

Также проверим данные в таблице user_info3.

```
cqlsh:ks_work1> select * from user_info3
... ;
```

country	id	birth	s	f_name	gender	l_name	num
CAN	7	1994	value7	bella	F	null	41
KOR	1	1995	value8	jason	M	jeong	18
KOR	5	1995	value8	lonan	M	kim	53
USA	2	1995	value6	kim	F	null	61
USA	3	1967	value6	scittt	M	park	32
USA	6	1967	value6	angela	F	park	25
USA	8	1985	value6	tommy	F	null	61
JPN	4	1974	value4	jane	F	null	null

(8 rows)

Рисунок 21 — Информация в user_info3

Также проверим записи в таблицах с 1 строкой на раздел (Рисунок 22).

```
cqlsh:ks_work1> select * from user_info2 per partition limit 1;
```

country	birth	id	f_name	gender	l_name	num	s
KOR	1995	1	jason	M	jeong	18	value8
USA	1967	3	scott	M	jeong	32	value6
USA	1974	4	jane	F	null	0	value4
USA	1985	2	tommy	M	park	19	value2
CAN	1994	7	bella	F	null	41	value7

(5 rows)

```
cqlsh:ks_work1> select * from user_info3 per partition limit 1;
```

country	id	birth	s	f_name	gender	l_name	num
CAN	7	1994	value7	bella	F	null	41
KOR	1	1995	value8	jason	M	jeong	18
USA	2	1995	value6	kim	F	null	61
JPN	4	1974	value4	jane	F	null	null

(4 rows)

Рисунок 22 — Данные со строкой на раздел

Выведем список лиц, включенных в KOR/JPN (Рисунок 23).

```
cqlsh:ks_work1> select f_name, l_name from user_info3 where country in ('KOR','JPN');
```

f_name	l_name
jane	null
jason	jeong
lonan	kim

(3 rows)

Рисунок 23 — Список лиц по фильтру

Найдем общее количество записей в таблице user_info3, а также сумму, минимальное и среднее значения поля num (Рисунок 24).

```
cqlsh:ks_work1> select count(id) as count, sum(num) as sum, min(num) as min, avg(num) as avg from user_info3;
```

count	sum	min	avg
8	291	18	41

(1 rows)

Warnings :
Aggregation query used without partition key

Рисунок 24 — Информация по таблице

Получим запись о стране KOR (Рисунок 25).

```
cqlsh:ks_work1> select * from user_info3 where country='KOR';
```

country	id	birth	s	f_name	gender	l_name	num
KOR	1	1995	value8	jason	M	jeong	18
KOR	5	1995	value8	lonan	M	kim	53

(2 rows)

Рисунок 25 — Записи о стране KOR

Выведем те записи, где страна KOR и num больше 30 (Рисунок 26).

```
cqlsh:ks_work1> select * from user_info3 where country='KOR' and num > 30 allow filtering;
```

country	id	birth	s	f_name	gender	l_name	num
KOR	5	1995	value8	lonan	M	kim	53

(1 rows)

Рисунок 26 — KOR и num больше 30

Заменяем значение s в KOR на 'value8' и сделаем его пустым через 30 секунд (Рисунок 27).

```
cqlsh:ks_work1> update user_info3 using ttl 30 set s='value_new' where country='KOR';
cqlsh:ks_work1> select * from user_info3;
```

country	id	birth	s	f_name	gender	l_name	num
CAN	7	1994	value7	bella	F	null	41
KOR	1	1995	value_new	jason	M	jeong	18
KOR	5	1995	value_new	lonan	M	kim	53
USA	2	1995	value6	kim	F	null	61
USA	3	1967	value6	scittt	M	park	32
USA	6	1967	value6	angela	F	park	25
USA	8	1985	value6	tommy	F	null	61
JPN	4	1974	value4	jane	F	null	null

Рисунок 27 — Изменение значения через время

Измененная таблица представлена на Рисунке 28.

```
cqlsh:ks_work1> select * from user_info3;
```

country	id	birth	s	f_name	gender	l_name	num
CAN	7	1994	value7	bella	F	null	41
KOR	1	1995	null	jason	M	jeong	18
KOR	5	1995	null	lonan	M	kim	53
USA	2	1995	value6	kim	F	null	61
USA	3	1967	value6	scittt	M	park	32
USA	6	1967	value6	angela	F	park	25
USA	8	1985	value6	tommy	F	null	61
JPN	4	1974	value4	jane	F	null	null

Рисунок 28 — Изменённая таблица

Обновим некоторые строки в таблице user_info3 (Рисунок 29).

```
cqlsh:ks_work1> insert into user_info3 (id, f_name, l_name, num, country, birth) values(4,'ann','kim', 85, 'JPN', '1974');
cqlsh:ks_work1> insert into user_info3 (id, l_name, f_name, country, birth, gender, s, num) values (3,'jeong','scott','USA','1967', 'M', 'value3', 32) if not exists;
```

[applied]	country	id	birth	s	f_name	gender	l_name	num
False	USA	3	1967	value6	scittt	M	park	32

```
cqlsh:ks_work1> UPDATE user_info3 SET f_name = 'scott' WHERE country = 'USA' and id = 3 and birth = '1967';
```

Рисунок 29 — Изменение в таблице

Удалим столбец s, в котором страна - США из таблицы user_info3 (Рисунок 30).

```
cqlsh:ks_work1> delete s from user_info3 where country='USA';
cqlsh:ks_work1> select * from user_info3;
```

country	id	birth	s	f_name	gender	l_name	num
CAN	7	1994	value7	bella	F	null	41
KOR	1	1995	null	jason	M	jeong	18
KOR	5	1995	null	lonan	M	kim	53
USA	2	1995	null	kim	F	null	61
USA	3	1967	null	scott	M	park	32
USA	6	1967	null	angela	F	park	25
USA	8	1985	null	tommy	F	null	61
JPN	4	1974	value4	ann	F	kim	85

(8 rows)

Рисунок 30 — Удаление столбца по ключу

Также удалим строки, в которых указана страна CAN или JPN, из таблицы user_info3 (Рисунок 31).

```
cqlsh:ks_work1> delete from user_info3 where country in ('CAN', 'JPN');
cqlsh:ks_work1> select * from user_info3;
```

country	id	birth	s	f_name	gender	l_name	num
KOR	1	1995	null	jason	M	jeong	18
KOR	5	1995	null	lonan	M	kim	53
USA	2	1995	null	kim	F	null	61
USA	3	1967	null	scott	M	park	32
USA	6	1967	null	angela	F	park	25
USA	8	1985	null	tommy	F	null	61

(6 rows)

Рисунок 31 — Удаление по ключу

В результате была показана работа со строками и ключами.

2.5 Самостоятельная работа

Создадим таблицу по соответствующим параметрам и заполним её информацией (Рисунок 32).

```
cqlsh:ks_work1> CREATE TABLE country (id uuid, name text, capital text, population int, PRIMARY KEY ((name, capital), id));
cqlsh:ks_work1> INSERT INTO country (id, name, capital, population) VALUES (uuid(), 'USA', 'Washington D.C.', 333098437);
cqlsh:ks_work1> INSERT INTO country (id, name, capital, population) VALUES (uuid(), 'Germany', 'Berlin', 84073352);
cqlsh:ks_work1> INSERT INTO country (id, name, capital, population) VALUES (uuid(), 'France', 'Paris', 65426179);
cqlsh:ks_work1> INSERT INTO country (id, name, capital, population) VALUES (uuid(), 'Korea', 'Seoul', 51305186);
```

Рисунок 32 — Создание таблицы

Заполненная таблица представлена на Рисунке 33.

```
cqlsh:ks_work1> select * from country;
```

name	capital	id	population
USA	Washington D.C.	20083a44-92b2-4ea0-9649-97a740e100c9	333098437
Korea	Seoul	dac4aa03-2c48-4828-99ab-b4968993dbca	51305186
France	Paris	85f708db-0402-4550-b3b4-19456a751713	65426179
Germany	Berlin	8063105b-1e04-4dc1-842f-3a7c375d3c04	84073352

(4 rows)

Рисунок 33 — Заполненная таблица

Запросим из таблицу информацию по населению Кореи (Рисунок 34).

```
cqlsh:ks_work1> SELECT name, population FROM country WHERE name = 'Korea' ALLOW FILTERING;
```

name	population
Korea	51305186

(1 rows)

Рисунок 34 — Население Кореи

Выведем население и столицы всех стран в таблице (Рисунок 35).

```
cqlsh:ks_work1> SELECT name, capital, population FROM country;
```

name	capital	population
USA	Washington D.C.	333098437
Korea	Seoul	51305186
France	Paris	65426179
Germany	Berlin	84073352

(4 rows)

Рисунок 35 — Население стран

Страны с населением менее 100000000 представлены на Рисунке 36.

```
cqlsh:ks_work1> SELECT name, capital, population FROM country WHERE population < 100000000 ALLOW FILTERING;
```

name	capital	population
Korea	Seoul	51305186
France	Paris	65426179
Germany	Berlin	84073352

(3 rows)

Рисунок 36 — Страны по фильтру

Покажем страны с населением от 50 до 100 миллионов человек (Рисунок 37).

```
cqlsh:ks_work1> SELECT name, capital, population FROM country WHERE population >= 50000000 AND population <= 100000000 ALLOW FILTERING;
```

name	capital	population
Korea	Seoul	51305186
France	Paris	65426179
Germany	Berlin	84073352

(3 rows)

Рисунок 37 — Страны по фильтру

Выведем страну с наименьшим населением (Рисунок 38).

```
cqlsh:ks_work1> SELECT name, population FROM country;
```

name	population
USA	333098437
Korea	51305186
France	65426179
Germany	84073352

(4 rows)

Рисунок 38 — Страны и население

В итоге страна с самым маленьким населением — Корея.

Выведем первую строку в таблице стран (Рисунок 39).

```
cqlsh:ks_work1> SELECT * FROM country LIMIT 1;
```

name	capital	id	population
USA	Washington D.C.	20083a44-92b2-4ea0-9649-97a740e100c9	333098437

Рисунок 39 — Первая страна в списке

Покажем общее количество стран (Рисунок 40).

```
cqlsh:ks_work1> SELECT COUNT(*) FROM country;
```

count
4

(1 rows)

Warnings :
Aggregation query used without partition key

Рисунок 40 — Общее количество стран

ВЫВОДЫ

В процессе практической работы были освоены ключевые этапы взаимодействия с Apache Cassandra: от установки и создания ключевых пространств до проектирования таблиц и выполнения операций чтения, записи, обновления и удаления данных. Полученный опыт подтвердил преимущества системы в сценариях, требующих высокой скорости записи, предсказуемой производительности и устойчивости к сбоям. Важным результатом стало понимание необходимости моделировать данные под конкретные шаблоны запросов, что позволяет эффективно использовать архитектурные особенности Cassandra и достигать заявленных характеристик масштабируемости и надёжности. Выполненные задания заложили основу для дальнейшего проектирования высоконагруженных приложений.